HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
_____ o0o _____

BÀI TẬP LỚN NHẬP MÔN HỌC SÂU

Tên đề tài: Phân loại bình luận độc hại

Nhóm lớp : 02
Số thứ tự nhóm: 21

Vũ Công Tuấn Dương MSV: B22DCKH024

HÀ NỘI, 05/2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1

1.1 Mục tiêu và phạm vi đề tài 1

1.2 Bố cục báo cáo 2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 5

2.1 Xử lý dữ liệu văn bản 5

2.2 Các mô hình Deep Learning 6

2.2.1 Long Short-Term Memory (LSTM) 6

2.2.2 Bidirectional LSTM (BiLSTM) 7

2.2.3 Gated Recurrent Unit (GRU) 8

2.2.4 Cơ chế Attention 9

2.2.5 Attention BiLSTM 10

2.2.6 Recurrent Convolutional Neural Network (RCNN) 10

2.2.7 Transformer Encoder 11

2.3 Các phương pháp tối ưu hóa huấn luyện 13

2.3.1 Xử lý dữ liệu mất cân bằng với Class-Weighted Loss 13

2.3.2 AdamW Optimizer với Weight Decay 14

2.3.3 Gradient Clipping 15

2.3.4 Warmup Cosine LR Scheduler 15

2.3.5 Early Stopping 16

2.3.6 Tìm kiếm Optimal Thresholds 17

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 18

3.1 Tập dữ liệu 18

3.2 Cấu hình thử nghiệm 18

3.3 So sánh tổng quan các mô hình 19

3.4 Kết quả chi tiết từng mô hình 21

3.4.1 Kết quả của RCNN 21

3.4.2 Kết quả của Attention BiLSTM 21

3.4.3 Kết quả của BiLSTM 22

3.4.4 Kết quả của Transformer 23

3.5 Phân tích ảnh hưởng của các kỹ thuật tối ưu hóa 23

3.6 Nhận xét chung 24

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 27

4.1 Tổng kết 27

4.2 Hướng phát triển 28

TÀI LIỆU THAM KHẢO 34

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

Bảng 3.1 So sánh hiệu suất các mô hình trên tập validation (159,571 mẫu) 20

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Mục tiêu và phạm vi đề tài

Trong bối cảnh phát triển không ngừng của mạng xã hội và các nền tảng giao tiếp trực tuyến, việc quản lý nội dung trở nên ngày càng quan trọng và cấp bách. Các bình luận độc hại, bao gồm những lời lẽ xúc phạm, đe dọa, phân biệt đối xử hay có nội dung tục tĩu, không chỉ ảnh hưởng tiêu cực đến trải nghiệm người dùng mà còn có thể gây ra những hậu quả nghiêm trọng về tâm lý cho những người phải tiếp xúc với chúng. Theo thống kê của các nghiên cứu gần đây, tỷ lệ người dùng internet từng gặp phải nạn quấy rối trực tuyến đã tăng đáng kể, đặc biệt trên các nền tảng mạng xã hội như Facebook, Twitter, YouTube và Reddit. Việc kiểm duyệt nội dung thủ công là một thách thức lớn khi khối lượng dữ liệu tăng lên theo cấp số nhân, đòi hỏi một giải pháp tự động hóa hiệu quả và có khả năng mở rộng. Các nghiên cứu trước đây như của Djuric và cộng sự (2015) đã chỉ ra rằng việc sử dụng các phương pháp học máy và học sâu để phát hiện nội dung độc hại là một hướng đi đầy tiềm năng [1]. Tương tự, Warner và cộng sự (2012) cũng đã nghiên cứu về việc phát hiện tự động sự quấy rối trên Twitter và nhận thấy rằng các phương pháp tự động có thể đạt hiệu quả cao trong việc nhận diện các hành vi vi phạm [2].

Mục tiêu chính của đề tài này là nghiên cứu và cài đặt từ gốc (from scratch) các mô hình Deep Learning khác nhau để giải quyết bài toán phân loại đa nhãn bình luận độc hại (Multi-label Toxic Comment Classification). Bài toán này được đặt ra với 6 nhãn cần dự đoán bao gồm: toxic (độc hại), severe_toxic (độc hại nghiêm trọng), obscene (tục tĩu), threat (đe dọa), insult (xúc phạm), và identity_hate (kỳ thị). Điểm đặc biệt của bài toán là một bình luận có thể thuộc nhiều hơn một nhãn cùng lúc, chẳng hạn một bình luận có thể vừa là toxic vừa là obscene, do đó đây là bài toán phân loại đa nhãn (multi-label classification) chứ không phải phân loại đa lớp (multi-class classification). Điều này đặt ra những thách thức riêng biệt trong việc thiết kế hàm mất mát và đánh giá mô hình, đòi hỏi phải sử dụng Binary Cross-Entropy thay vì Categorical Cross-Entropy như trong các bài toán phân loại đa lớp truyền thống.

Về phạm vi nghiên cứu, đề tài tập trung vào việc cài đặt và so sánh bảy mô hình Deep Learning khác nhau, bao gồm: Long Short-Term Memory (LSTM) [3], Bidirectional LSTM (BiLSTM), Gated Recurrent Unit (GRU) [4], Recurrent Convolutional Neural Network (RCNN) [5], Transformer Encoder [6], Attention LSTM, và Attention BiLSTM. Tất cả các mô hình này đều được cài đặt từ đầu mà không sử dụng các thư viện có sẵn như nn.LSTM hay nn.GRU của PyTorch, nhằm mục đích

thể hiện sự hiểu biết sâu về kiến trúc bên trong của các mô hình này. Việc cài đặt từ gốc bao gồm việc xây dựng các cell (ô) tính toán như LSTM Cell và GRU Cell từ các phép toán ma trận cơ bản, cơ chế Attention từ đầu, các lớp Self-Attention và Multi-Head Attention cho Transformer, cũng như các chiến lược Pooling khác nhau như Global Average Pooling, Global Max Pooling, và Attention Pooling.

Đề tài cũng áp dụng các kỹ thuật tối ưu hóa quan trọng để cải thiện hiệu suất mô hình và khả năng hội tụ. Class-Weighted Loss được sử dụng để xử lý dữ liệu mất cân bằng nghiêm trọng giữa các lớp [7]. AdamW Optimizer với Weight Decay được áp dụng để cải thiện khả năng tổng quát hóa của mô hình [8]. Gradient Clipping được sử dụng để ngăn chặn exploding gradient trong quá trình huấn luyện các mô hình RNN [9]. Warmup Cosine LR Scheduler được áp dụng để huấn luyện ổn định và hội tụ tốt hơn. Early Stopping được sử dụng để ngăn overfitting tự động dựa trên validation ROC-AUC. Ngoài ra, kỹ thuật tìm kiếm Optimal Thresholds riêng biệt cho từng lớp cũng được áp dụng để cải thiện precision và recall.

Để thực hiện các thử nghiệm, đề tài sử dụng bộ dữ liệu Jigsaw Toxic Comment Classification Challenge từ Kaggle [10], một trong những bộ dữ liệu chuẩn và phổ biến nhất trong lĩnh vực phân loại văn bản độc hại. Bộ dữ liệu này chứa khoảng 160 nghìn bình luận từ Wikipedia với các nhãn được gán thủ công bởi các chuyên gia kiểm duyệt, cung cấp một bài toán thực tế và đầy thách thức cho việc nghiên cứu các phương pháp Deep Learning trong xử lý ngôn ngữ tự nhiên. Sự mất cân bằng nghiêm trọng giữa các lớp (từ 0.2% cho đến 9.4%) tạo ra một thách thức thực sự để kiểm chứng hiệu quả của các kỹ thuật xử lý dữ liệu mất cân bằng.

Bên cạnh đó, đề tài cũng sử dụng bộ tokenizer của DistilBERT (distilbert-base-uncased) từ thư viện Hugging Face Transformers [11] để chuyển đổi văn bản thành các token IDs. DistilBERT là phiên bản nhỏ gọn hơn và nhanh hơn của BERT, được huấn luyện bằng knowledge distillation và đạt được 97% hiệu suất của BERT trên benchmark GLUE trong khi chỉ sử dụng 60% tham số [12]. Việc sử dụng pre-trained tokenizer thay vì cài đặt tokenizer từ gốc giúp đảm bảo rằng các token được tách một cách thông minh, hiểu được subword tokenization và xử lý tốt các từ hiếm gặp.

1.2 Bố cục báo cáo

Báo cáo này được tổ chức thành bốn chương chính để trình bày một cách có hệ thống toàn bộ quá trình nghiên cứu và kết quả đạt được.

Chương 1 giới thiệu tổng quan về đề tài, bao gồm bối cảnh thực tiễn về vấn đề bình luận độc hại trên mạng xã hội và tầm quan trọng của việc tự động hóa quá trình kiểm duyệt nội dung. Phần này cũng trình bày rõ mục tiêu, phạm vi nghiên

cứu của đề tài, bao gồm danh sách 7 mô hình Deep Learning được cài đặt và so sánh, cũng như các kỹ thuật tối ưu hóa được áp dụng. Cuối chương giới thiệu cấu trúc của báo cáo và mối liên hệ giữa các chương.

Chương 2 trình bày cơ sở lý thuyết liên quan đến các mô hình Deep Learning được cài đặt trong đề tài. Phần đầu của chương tập trung vào các mô hình RNN (Recurrent Neural Network) cơ bản bao gồm LSTM với cơ chế các cổng (gates) để giải quyết vấn đề vanishing gradient, theo đúng như bài báo gốc của Hochreiter và Schmidhuber năm 1997 [3]; GRU với thiết kế đơn giản hơn sử dụng 2 cổng như trong nghiên cứu của Cho và cộng sự năm 2014 [4]; và BiLSTM cho phép xử lý hai chiều để nắm bắt ngữ cảnh toàn diện. Phần tiếp theo trình bày về cơ chế Attention (chú ý), được giới thiệu ban đầu trong bài toán Neural Machine Translation bởi Bahdanau và cộng sự năm 2014 [13], cũng như cách kết hợp Attention với LSTM và BiLSTM để tạo ra các mô hình Attention LSTM và Attention BiLSTM. Tiếp theo là phần giới thiệu về RCNN (Recurrent Convolutional Neural Network), sự kết hợp giữa RNN và CNN theo nghiên cứu của Lai và cộng sự năm 2015 [5], và cuối cùng là Transformer Encoder với cơ chế Self-Attention và Positional Encoding như trong bài báo "Attention Is All You Need" của Vaswani và cộng sự năm 2017 [6]. Ngoài ra, chương này cũng trình bày chi tiết các kỹ thuật tối ưu hóa đã áp dụng bao gồm Class-Weighted Loss, AdamW, Gradient Clipping, Warmup Cosine LR Scheduler, và Early Stopping, kèm theo cơ sở toán học và lý do sử dụng của từng kỹ thuật.

Chương 3 trình bày chi tiết về các thử nghiệm và kết quả đánh giá. Phần đầu mô tả về bộ dữ liệu Jigsaw, bao gồm phân tích phân bố của 6 lớp nhãn và mức độ mất cân bằng của dữ liệu. Phần tiếp theo trình bày cấu hình thử nghiệm, bao gồm các siêu tham số (hyperparameters) huấn luyện, cấu hình phần cứng (GPU NVIDIA H100), và các thư viện được sử dụng (PyTorch [14], Hugging Face Transformers [11]). Phần trung tâm trình bày kết quả so sánh giữa 7 mô hình theo các tiêu chí khác nhau như ROC-AUC, F1 Score (macro và micro), Precision và Recall, sử dụng các độ đo được phân tích chi tiết dựa trên nghiên cứu của Fawcett (2006) về ROC Analysis [15] và Sokolova và Lapalme (2009) về các độ đo cho bài toán phân loại [16]. Cuối chương phân tích chi tiết kết quả của từng mô hình, nhận xét ưu nhược điểm, và phân tích ảnh hưởng của các kỹ thuật tối ưu hóa đến hiệu suất cuối cùng.

Chương 4 tổng kết toàn bộ đề tài, trình bày các kết luận rút ra từ quá trình nghiên cứu, đánh giá mức độ đạt được của các mục tiêu ban đầu, và đề xuất các hướng phát triển trong tương lai để có thể cải thiện thêm hiệu quả của các mô hình phân loại bình luận độc hại. Các hướng phát triển được đề xuất bao gồm: sử dụng

pretrained embeddings như GloVe [17] hoặc FastText [18] thay vì embedding ngẫu nhiên, fine-tuning class weights với các phương pháp tối ưu hóa nâng cao, xây dựng ensemble model kết hợp nhiều mô hình, data augmentation cho các lớp thiểu số, hyperparameter tuning chi tiết cho từng mô hình, và tiềm năng của các mô hình Transformer-based pretrained như BERT [19] hoặc RoBERTa.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Xử lý dữ liệu văn bản

Việc xử lý dữ liệu văn bản đóng vai trò nền tảng trong bất kỳ bài toán xử lý ngôn ngữ tự nhiên nào, bao gồm cả bài toán phân loại bình luận độc hại. Theo cuốn sách "Modern Information Retrieval" của Baeza-Yates và Ribeiro-Neto (1999), tiền xử lý văn bản là bước quan trọng để chuẩn hóa dữ liệu đầu vào và loại bỏ nhiễu, giúp cải thiện đáng kể chất lượng của các mô hình phân loại [20].

Quá trình tokenize bắt đầu bằng việc chuyển đổi văn bản gốc thành danh sách các token IDs, đồng thời tạo ra attention mask để đánh dấu các token thực sự so với các token padding. Mỗi câu được thêm các token đặc biệt là [CLS] (classification) ở đầu và [SEP] (separator) ở cuối để phân tách các câu khi cần thiết. Độ dài tối đa của chuỗi được giới hạn ở 128 tokens, giúp cân bằng giữa khả năng nắm bắt ngữ cảnh và tốc độ huấn luyện. Attention mask sẽ có giá trị 1 cho các token thực và 0 cho các token padding, cho phép mô hình bỏ qua phần padding khi tính toán.

Trong bài toán phân loại đa nhãn, mỗi bình luận được gán nhãn bởi 6 cột binary tương ứng với 6 loại độc hại: toxic, severe_toxic, obscene, threat, insult, và identity_hate. Điều đáng chú ý là một bình luận có thể đồng thời thuộc nhiều nhãn. Điều này phân biệt bài toán multi-label classification với multi-class classification, đòi hỏi việc sử dụng Binary Cross-Entropy loss thay vì Categorical Cross-Entropy [21]. Hàm mất mát BCEWithLogitsLoss được tính như sau:

$$\mathcal{L}_{BCE} = -\frac{1}{N \times C} \sum_{i=1}^{N} \sum_{c=1}^{C} \left[y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic})) \right] \tag{2.1}$$

Trong đó N là số mẫu, C là số lớp (6), y_{ic} là nhãn thực của mẫu i lớp c, x_{ic} là logits đầu ra của mô hình, và σ là hàm sigmoid. Khi sử dụng class weights (pos_weight), công thức trở thành:

$$\mathcal{L}_{BCE}^{weighted} = -\frac{1}{N \times C} \sum_{i=1}^{N} \sum_{c=1}^{C} w_c \left[y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic})) \right] \tag{2.2}$$

Với w_c là trọng số của lớp c, được tính bằng tỷ lệ $\frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c}$.

2.2 Các mô hình Deep Learning

2.2.1 Long Short-Term Memory (LSTM)

Mạng Long Short-Term Memory (LSTM) là một biến thể của mạng RNN (Recurrent Neural Network) được giới thiệu bởi Hochreiter và Schmidhuber vào năm 1997, nhằm giải quyết hai vấn đề cơ bản của RNN truyền thống: vanishing gradient và exploding gradient [3]. Khi chuỗi đầu vào quá dài, gradient trong quá trình backpropagation through time (BPTT) có xu hướng giảm rất nhanh hoặc tăng rất nhanh, khiến mô hình không thể học được các phụ thuộc từ xa trong chuỗi.

Kiến trúc cốt lõi của LSTM dựa trên khái niệm cell state (trạng thái ô), đóng vai trò như một "băng truyền" cho phép thông tin chảy dọc theo chuỗi mà hầu như không bị thay đổi. Cell state được điều khiển bởi ba cổng (gates): Forget Gate, Input Gate, và Output Gate.

Các công thức toán học của LSTM được biểu diễn như sau:

$$f_t = \sigma(W_{hf}h_{t-1} + W_{xf}x_t + b_f) \quad \text{(Forget Gate)} \tag{2.3}$$

Forget Gate quyết định thông tin nào từ cell state trước đó cần được loại bỏ. Hàm sigmoid σ đưa ra giá trị trong khoảng [0,1], trong đó 0 có nghĩa là "loại bỏ hoàn toàn" và 1 có nghĩa là "giữ lại hoàn toàn".

$$i_t = \sigma(W_{hi}h_{t-1} + W_{xi}x_t + b_i) \quad \text{(Input Gate)} \tag{2.4}$$

Input Gate quyết định thông tin mới nào sẽ được thêm vào cell state.

$$\tilde{c}_t = \tanh(W_{hc}h_{t-1} + W_{xc}x_t + b_c) \quad \text{(Candidate Cell State)} \tag{2.5}$$

Candidate Cell State tạo ra các giá trị ứng viên để thêm vào cell state. Hàm tanh đưa ra giá trị trong khoảng [-1,1], cho phép cả giá trị dương và âm.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(Cell State Update)} \tag{2.6}$$

Cell State mới được cập nhật bằng cách kết hợp thông tin cũ (được điều khiển bởi Forget Gate) và thông tin mới (được điều khiển bởi Input Gate). Toán tử $\odot$ là phép nhân Hadamard (element-wise).

$$o_t = \sigma(W_{ho}h_{t-1} + W_{xo}x_t + b_o) \quad \text{(Output Gate)} \tag{2.7}$$

Output Gate quyết định phần nào của cell state sẽ được xuất ra.

$$h_t = o_t \odot \tanh(c_t) \quad \text{(Hidden State)} \tag{2.8}$$

Hidden State mới là kết quả của việc nhân Output Gate với tanh của Cell State, quyết định thông tin nào được truyền ra ngoài.

Trong đề tài này, LSTM được cài đặt từ gốc sử dụng các tham số nn.Parameter và các phép toán ma trận cơ bản. Mô hình bao gồm một lớp Embedding chuyển đổi token IDs thành các vector có chiều EMBEDDING_DIM, tiếp theo là các tầng LSTM xử lý tuần tự từng token, và cuối cùng là Global Average Pooling để tổng hợp hidden states từ tất cả các time steps thành một vector duy nhất:

$$\bar{h} = \frac{1}{T} \sum_{t=1}^{T} h_t \tag{2.9}$$

2.2.2 Bidirectional LSTM (BiLSTM)

Bidirectional LSTM (BiLSTM) là một mở rộng của LSTM, cho phép xử lý chuỗi đầu vào theo cả hai hướng cùng lúc: từ trái sang phải (forward) và từ phải sang trái (backward) [9]. Ý tưởng cốt lõi là mỗi từ trong câu không chỉ phụ thuộc vào ngữ cảnh đứng trước nó mà còn phụ thuộc vào ngữ cảnh đứng sau nó. Ví dụ trong câu "I don’t hate this movie", từ "hate" mang nghĩa tiêu cực, nhưng trong ngữ cảnh đầy đủ "don’t hate" lại mang nghĩa tích cực.

Về mặt kỹ thuật, BiLSTM bao gồm hai mạng LSTM chạy song song. Công thức tính hidden state của BiLSTM tại mỗi vị trí t:

$$\overrightarrow{h_t} = \text{LSTM}_{\rightarrow}(x_t, \overrightarrow{h_{t-1}}) \quad \text{(Forward)} \tag{2.10}$$

$$\overleftarrow{h_t} = \text{LSTM}_{\leftarrow}(x_t, \overleftarrow{h_{t+1}}) \quad \text{(Backward)} \tag{2.11}$$

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}] \quad \text{(Concatenation)} \tag{2.12}$$

Hidden state của BiLSTM được tạo bằng cách concatenate hidden state từ forward LSTM và hidden state từ backward LSTM tại vị trí đó. Do đó, kích thước hidden state của BiLSTM gấp đôi so với LSTM thông thường: $\text{hidden_dim}_{BiLSTM} = 2 \times \text{hidden_dim}_{LSTM}$.

Pooling cuối cùng sử dụng Global Average Pooling để tổng hợp thông tin từ tất cả các time steps. Với BiLSTM, mỗi hidden state đã có chiều gấp đôi, nên vector đầu ra sau pooling có kích thước hidden_size $\times$ 2.

2.2.3 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) là một biến thể khác của RNN, được đề xuất bởi Cho vào năm 2014 trong bài báo về RNN Encoder-Decoder cho dịch máy thống kê [4], với thiết kế đơn giản hóa hơn so với LSTM. GRU sử dụng ít cổng hơn (chỉ 2 cổng thay vì 3) và không sử dụng cell state riêng biệt, chỉ dùng hidden state duy nhất, giúp giảm số lượng tham số cần học khoảng 25%.

Các công thức toán học của GRU:

$$z_t = \sigma(W_{hz}h_{t-1} + W_{xz}x_t + b_z) \quad \text{(Update Gate)} \tag{2.13}$$

Update Gate z_t kết hợp vai trò của Forget Gate và Input Gate trong LSTM. Khi z_t gần 1, mô hình giữ lại hầu hết thông tin cũ; khi z_t gần 0, mô hình thêm nhiều thông tin mới.

$$r_t = \sigma(W_{hr}h_{t-1} + W_{xr}x_t + b_r) \quad \text{(Reset Gate)} \tag{2.14}$$

Reset Gate r_t kiểm soát mức độ thông tin quá khứ cần được bỏ qua khi tính toán candidate hidden state mới. Nếu r_t gần 0, mô hình bỏ qua hoàn toàn hidden state trước đó.

$$\tilde{h}_t = \tanh(W_{hn}(r_t \odot h_{t-1}) + W_{xn}x_t + b_n) \quad \text{(Candidate Hidden State)} \tag{2.15}$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \quad \text{(Hidden State Update)} \tag{2.16}$$

Hidden state mới là sự nội suy giữa candidate state và hidden state trước đó, được điều khiển bởi update gate. Công thức này cho phép mô hình linh hoạt quyết định bao nhiêu thông tin cũ cần được giữ lại.

Ưu điểm chính của GRU so với LSTM là ít tham số hơn, dẫn đến huấn luyện nhanh hơn và ít nguy cơ overfitting hơn. Trong nhiều thử nghiệm thực tế, hiệu suất của GRU và LSTM thường tương đương nhau, và sự lựa chọn giữa hai kiến trúc phụ thuộc vào đặc điểm cụ thể của bài toán và dữ liệu.

2.2.4 Cơ chế Attention

Cơ chế Attention (chú ý) là một trong những đột phá quan trọng nhất trong lĩnh vực Deep Learning, được giới thiệu ban đầu trong bài toán dịch máy Neural Machine Translation bởi Bahdanau và cộng sự vào năm 2014 [13]. Ý tưởng cốt lõi của Attention là thay vì dựa hoàn toàn vào hidden state cuối cùng để đại diện cho toàn bộ chuỗi (như trong RNN truyền thống), mô hình có thể "chú ý" (tập trung) vào các phần khác nhau của chuỗi đầu vào với mức độ khác nhau.

Trong bài toán phân loại bình luận độc hại, không phải từ nào cũng quan trọng như nhau. Ví dụ trong câu "You are an idiot", từ "idiot" mang ý nghĩa quan trọng nhất để xác định bình luận là toxic, trong khi các từ như "are", "an" gần như không cung cấp thông tin hữu ích.

Công thức toán học của Self-Attention được sử dụng trong đề tài:

$$e_t = \tanh(W_a h_t + b_a) \quad \text{(Energy Score)} \tag{2.17}$$

Bước đầu tiên tính energy scores e_t cho mỗi time step, trong đó h_t là hidden state tại time step t và W_a là ma trận trọng số có thể huấn luyện. Energy score đại diện cho mức độ "quan trọng" của mỗi time step.

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)} = \text{softmax}(e_t) \quad \text{(Attention Weights)} \tag{2.18}$$

Attention weights α_t được tính bằng softmax của các energy scores, đảm bảo tổng các trọng số bằng 1 ($\sum_{t=1}^{T} \alpha_t = 1$). Điều này cho phép interpretability: ta có thể biết được mô hình đang tập trung vào phần nào của văn bản.

$$c = \sum_{t=1}^{T} \alpha_t h_t \quad \text{(Context Vector)} \tag{2.19}$$

Context vector c là tổng có trọng số của tất cả hidden states. Context vector này được sử dụng thay cho mean pooling hoặc hidden state cuối cùng làm đầu vào cho lớp phân loại. Điểm khác biệt chính so với mean pooling ($\bar{h} = \frac{1}{T} \sum h_t$) là attention cho phép mô hình học để tập trung vào các phần quan trọng.

Yang và cộng sự (2016) đã phát triển kiến trúc Hierarchical Attention Networks sử dụng attention ở nhiều cấp độ và đạt kết quả ấn tượng trên bài toán phân loại văn bản [22].

2.2.5 Attention BiLSTM

Attention BiLSTM là sự kết hợp giữa kiến trúc BiLSTM và cơ chế Attention, tận dụng ưu điểm của cả hai. BiLSTM cung cấp khả năng nắm bắt ngữ cảnh hai chiều, trong khi Attention cho phép mô hình tập trung vào các từ quan trọng nhất khi đưa ra quyết định phân loại.

Trong mô hình này, đầu ra của BiLSTM là một chuỗi các hidden states (mỗi hidden state có chiều gấp đôi do concatenate hai hướng), sau đó Self-Attention được áp dụng lên chuỗi này để tạo ra một context vector duy nhất đại diện cho toàn bộ câu. Attention weights được tính dựa trên các hidden states từ BiLSTM, cho phép mô hình học được tổ hợp ngữ cảnh hai chiều của từng từ trước khi quyết định mức độ quan trọng.

Mô hình Attention BiLSTM đạt được kết quả F1 Macro tốt nhất trong số tất cả các mô hình được thử nghiệm, chứng tỏ rằng sự kết hợp giữa ngữ cảnh hai chiều và cơ chế attention thực sự mang lại hiệu quả vượt trội. Attention giúp cải thiện đáng kể precision (từ 0.36 lên 0.47) trong khi vẫn duy trì recall ở mức cao (0.84), cho thấy mô hình đã học được cách phân biệt tốt hơn giữa các trường hợp thực sự thuộc nhãn độc hại và các trường hợp dương tính giả.

Số lượng tham số của Attention BiLSTM là 6.5 triệu, gần tương đương với BiLSTM cơ bản (6.3 triệu), cho thấy attention layer chỉ thêm một lượng nhỏ tham số nhưng mang lại cải thiện đáng kể về hiệu suất.

2.2.6 Recurrent Convolutional Neural Network (RCNN)

RCNN (Recurrent Convolutional Neural Network) là kiến trúc kết hợp giữa RNN và CNN, được đề xuất bởi Lai và cộng sự vào năm 2015 cho bài toán phân loại văn bản tại hội nghị AAAI [5]. Ý tưởng cốt lõi của RCNN là sử dụng BiLSTM để thu thập ngữ cảnh hai chiều cho mỗi từ, sau đó kết hợp ngữ cảnh đó với embedding gốc của từ để tạo ra biểu diễn phong phú hơn, rồi cuối cùng sử dụng Max Pooling thay vì Average Pooling để trích xuất các đặc trưng quan trọng nhất.

Kiến trúc RCNN trong đề tài bao gồm các bước sau:

Bước 1: BiLSTM tạo ngữ cảnh hai chiều

$$c_L(t) = \overrightarrow{h_t} \quad \text{(Ngữ cảnh trái tại vị trí } t\text{)} \tag{2.20}$$

$$c_R(t) = \overleftarrow{h_t} \quad \text{(Ngữ cảnh phải tại vị trí } t\text{)} \tag{2.21}$$

Bước 2: Concatenate ngữ cảnh với embedding

$$x_t^{combined} = [c_L(t); e_t; c_R(t)] \quad \text{(Biểu diễn kết hợp)} \tag{2.22}$$

Vector biểu diễn tại mỗi vị trí được tạo bằng cách concatenate ngữ cảnh trái, embedding gốc, và ngữ cảnh phải. Kích thước của vector này là: dim(c_L)+dim(e_t)+ dim$(c_R) = 2 \times$ hidden_size + embedding_dim.

Bước 3: Fusion layer với tanh

$$y_t = \tanh(W_f \cdot x_t^{combined} + b_f) \quad \text{(Fusion)} \tag{2.23}$$

Fusion layer tích hợp các đặc trưng từ ba nguồn (ngữ cảnh trái, embedding, ngữ cảnh phải) thành một vector biểu diễn duy nhất cho mỗi vị trí.

Bước 4: Max Pooling

$$\hat{y} = \max_{t \in [1,T]} y_t \quad \text{(Global Max Pooling)} \tag{2.24}$$

Global Max Pooling lấy giá trị lớn nhất theo chiều thời gian, giúp tập trung vào những đặc trưng nổi bật nhất. Max pooling đặc biệt hiệu quả khi các từ độc hại (như "fuck", "shit" cho obscene) xuất hiện rõ ràng và max pooling có thể dễ dàng chọn ra chúng.

RCNN đạt được ROC-AUC cao nhất trong số tất cả các mô hình (0.9839), cho thấy khả năng xếp hạng các mẫu dương tính cao hơn các mẫu âm tính của mô hình này là tốt nhất. Đặc biệt, RCNN đạt hiệu suất rất cao trên nhãn obscene (F1 = 0.81 với optimal thresholds).

Tuy nhiên, RCNN có số lượng tham số nhiều nhất (6.6 triệu) và F1 Macro của nó (0.5689) không cao bằng Attention BiLSTM (0.6311), cho thấy max pooling không cân bằng precision/recall tốt bằng attention.

2.2.7 Transformer Encoder

Transformer là kiến trúc được giới thiệu trong bài báo "Attention Is All You Need" của Vaswani và cộng sự vào năm 2017 tại hội nghị NeurIPS, đánh dấu một bước tiến lớn trong lĩnh vực xử lý ngôn ngữ tự nhiên [6]. Điểm khác biệt cốt lõi của Transformer so với các mô hình RNN là nó không sử dụng cơ chế tuần tự (sequential processing), mà thay vào đó xử lý toàn bộ chuỗi đầu vào song song trong một bước duy nhất.

Do Transformer không xử lý tuần tự nên nó không có thông tin về vị trí của các từ trong câu một cách tự nhiên. Positional Encoding được sử dụng để bổ sung thông tin vị trí:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad \text{(Vị trí chẵn)} \tag{2.25}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad \text{(Vị trí lẻ)} \tag{2.26}$$

Trong đó pos là vị trí của từ, i là chiều của embedding, và d là tổng chiều của embedding. Các hàm sine và cosine với các tần số khác nhau tạo ra một vector duy nhất cho mỗi vị trí mà mô hình có thể học được.

Self-Attention là cơ chế cốt lõi của Transformer. Công thức attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.27}$$

Trong đó Q (Query), K (Key), và V (Value) là ba ma trận được tính từ đầu vào: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Hệ số $\sqrt{d_k}$ trong mẫu số giúp ổn định gradient trong quá trình huấn luyện khi d_k lớn.

Multi-Head Attention chạy nhiều attention heads song song:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.28}$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i) \tag{2.29}$$

Mỗi head với một không gian con khác nhau, cho phép mô hình học các loại quan hệ khác nhau đồng thời: head này có thể học quan hệ ngữ pháp, head kia học quan hệ ngữ nghĩa, và một head khác học các mẫu phủ định.

Transformer Encoder Block bao gồm hai thành phần chính với Residual Connection và Layer Normalization:

$$x' = \text{LayerNorm}(x + \text{MultiHead}(x, x, x)) \tag{2.30}$$

$$x'' = \text{LayerNorm}(x' + \text{FFN}(x')) \tag{2.31}$$

Trong đó Feed-Forward Network (FFN) gồm hai lớp tuyến tính với ReLU activation:

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2 \tag{2.32}$$

Layer Normalization [23] được tính như sau:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{2.33}$$

Trong đó $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$ và $\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$ lần lượt là mean và variance của input. Các tham số γ và β là các tham số có thể học được.

Kết quả của Transformer trong thử nghiệm cho thấy mô hình này đạt ROC-AUC ở mức 0.9789 nhưng F1 Macro thấp nhất trong tất cả các mô hình (0.4625). Nguyên nhân có thể bao gồm việc sử dụng embedding ngẫu nhiên thay vì pretrained embeddings [17], [18], và cấu hình (2 layers, 8 heads) có thể chưa đủ mạnh.

2.3 Các phương pháp tối ưu hóa huấn luyện

2.3.1 Xử lý dữ liệu mất cân bằng với Class-Weighted Loss

Một trong những thách thức lớn nhất của bài toán phân loại bình luận độc hại là sự mất cân bằng nghiêm trọng giữa các lớp. Theo phân tích dữ liệu từ bộ Jigsaw [10], chỉ khoảng 9.4% bình luận thuộc lớp toxic, trong khi các lớp thiểu số như threat chỉ chiếm khoảng 0.2%.

Để giải quyết vấn đề này, đề tài sử dụng Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) với trọng số lớp (pos_weight). Trọng số cho mỗi lớp được tính bằng:

$$w_c = \frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c} = \frac{N - p_c}{p_c} \tag{2.34}$$

Trong đó p_c là số mẫu dương tính của lớp c và N là tổng số mẫu. Trọng số này nhân với loss của các mẫu dương, khiến việc bỏ sót một mẫu độc hại có chi phí cao hơn nhiều so với việc dự đoán sai một mẫu không độc hại.

Để tránh trọng số quá lớn gây ra over-penalizing lớp đa số, đề tài áp dụng clipping:

$$w_c^{clipped} = \min(w_c, 50) \tag{2.35}$$

Các trọng số cuối cùng: toxic = 9.43, severe_toxic = 50.00 (clip), obscene = 17.89, threat = 50.00 (clip), insult = 19.26, identity_hate = 50.00 (clip).

Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng [7]:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{2.36}$$

Trong đó p_t là xác suất của lớp đúng, α_t là trọng số cân bằng, và γ là focusing parameter. Focal Loss giảm trọng số của các mẫu dễ phân loại và tập trung vào các mẫu khó.

Sokolova và Lapalme (2009) đã phân tích chi tiết các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score đặc biệt hữu ích khi dữ liệu mất cân bằng [16]:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.37}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{2.38}$$

F1 Macro là trung bình F1 của các lớp: $F1_{macro} = \frac{1}{C}\sum_{c=1}^{C} F1_c$.

2.3.2 AdamW Optimizer với Weight Decay

Adam (Adaptive Moment Estimation) được giới thiệu bởi Kingma và Ba năm 2015 [24], kết hợp Momentum và RMSprop:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{(First moment estimate)} \tag{2.39}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{(Second moment estimate)} \tag{2.40}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(Bias correction)} \tag{2.41}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad \text{(Parameter update)} \tag{2.42}$$

AdamW (Adam with Decoupled Weight Decay) được đề xuất bởi Loshchilov và Hutter năm 2017 [8], tách biệt weight decay khỏi gradient update:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \cdot \lambda \cdot \theta_t \tag{2.43}$$

Trong đó λ là weight decay coefficient (10^{-4} trong đề tài). Việc tách biệt weight decay giúp đảm bảo regularization effect nhất quán bất kể giá trị của adaptive terms.

Weight decay (L2 regularization) đóng vai trò quan trọng trong việc ngăn chặn overfitting. Nó khuyến khích các weights có giá trị nhỏ, giúp cải thiện khả năng tổng quát hóa của mô hình [21].

Srivastava và cộng sự (2014) đã nghiên cứu về Dropout như một phương pháp

regularization hiệu quả khác [25]. Trong đề tài, dropout với tỷ lệ 0.3 được áp dụng trước lớp fully connected cuối cùng để ngăn overfitting.

2.3.3 Gradient Clipping

Gradient Clipping là kỹ thuật quan trọng đặc biệt đối với các mô hình RNN. Trong quá trình backpropagation through time, gradient được nhân qua nhiều time steps liên tiếp, có thể dẫn đến exploding gradient.

Kỹ thuật Gradient Clipping giới hạn giá trị của gradient bằng cách rescale nó nếu norm vượt quá ngưỡng:

$$\text{if } \|g\|_2 > \text{max_norm}: \quad g \leftarrow g \cdot \frac{\text{max_norm}}{\|g\|_2} \tag{2.44}$$

Trong đó $\|g\|_2 = \sqrt{\sum_i g_i^2}$ là L2 norm của gradient. Trong đề tài, max_norm = 1.0 được sử dụng. Gradient clipping được áp dụng sau khi gọi loss.backward() và trước khi gọi optimizer.step().

Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài [9].

LeCun và cộng sự (1998) đã thảo luận chi tiết về các thủ thuật huấn luyện mạng nơ-ron, bao gồm tầm quan trọng của việc kiểm soát gradient để đảm bảo quá trình huấn luyện ổn định [26].

2.3.4 Warmup Cosine LR Scheduler

Learning rate là một trong những siêu tham số quan trọng nhất trong huấn luyện mô hình Deep Learning. Đề tài sử dụng Warmup Cosine LR Scheduler - sự kết hợp của hai chiến lược điều chỉnh learning rate.

Giai đoạn Warmup (10% tổng số steps đầu tiên):

$$lr(t) = lr_{base} \cdot \frac{t}{T_{warmup}}, \quad 0 \leq t < T_{warmup} \tag{2.45}$$

Learning rate tăng tuyến tính từ 0 lên lr_{base} (2×10^{-3}), giúp mô hình ổn định dần dần và tránh các cập nhật quá mạnh ở giai đoạn đầu.

Giai đoạn Cosine Decay (90% còn lại):

$$lr(t) = lr_{min} + (lr_{max} - lr_{min}) \cdot \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_{warmup}}{T_{total} - T_{warmup}}\right)\right) \tag{2.46}$$

Learning rate giảm dần theo hàm cosine, với $lr_{min} = 0$ và $lr_{max} = lr_{base}$. Cosine decay duy trì learning rate ở mức trung bình trong thời gian dài hơn so với linear decay, cho phép mô hình có đủ thời gian để khám phá không gian tham số.

Vaswani và cộng sự (2017) đã sử dụng thành công warmup schedule trong huấn luyện Transformer và nhận thấy rằng nó giúp ổn định quá trình huấn luyện trong giai đoạn đầu [6].

Loshchilov và Hutter (2017) đã nghiên cứu về SGDR (Stochastic Gradient Descent with Warm Restarts) và nhận thấy rằng việc sử dụng cosine annealing với warm restarts mang lại cải thiện đáng kể trong nhiều bài toán [8].

2.3.5 Early Stopping

Early Stopping là kỹ thuật regularization đơn giản nhưng hiệu quả, nhằm ngăn chặn overfitting bằng cách dừng quá trình huấn luyện khi validation metric không còn cải thiện.

Đề tài sử dụng Early Stopping với cấu hình:

- **patience** = 2 epochs
- **mode** = 'max' (theo dõi ROC-AUC)
- **Metric** = ROC-AUC (thay vì loss)

ROC-AUC (Receiver Operating Characteristic - Area Under Curve) đo lường khả năng phân biệt của mô hình giữa các lớp dương tính và âm tính:

$$\text{AUC} = \int_0^1 \text{TPR}(t)\, d\text{FPR}(t) \tag{2.47}$$

Trong đó TPR (True Positive Rate) = Recall và FPR (False Positive Rate) = $\frac{FP}{FP+TN}$. AUC bằng 0.5 có nghĩa là dự đoán ngẫu nhiên, và AUC bằng 1.0 có nghĩa là phân loại hoàn hảo [15].

Lý do sử dụng ROC-AUC thay vì loss là vì loss có thể bị ảnh hưởng bởi class weights, trong khi ROC-AUC đo lường khả năng phân biệt thực sự của mô hình một cách threshold-independent. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng các metric phù hợp để tránh overfitting [27].

Thuật toán Early Stopping:

```
Input: patience p, best_score, current_score
if current_score > best_score then
    best_score ← current_score
    patience_counter ← 0
    Lưu model checkpoint
else
    patience_counter ← patience_counter + 1
    if patience_counter ≥ p then
        Khôi phục model từ checkpoint tốt nhất
        Dừng huấn luyện
```

Wolpert (1997) đã chứng minh No Free Lunch Theorems, cho thấy rằng không có thuật toán tối ưu nhất cho mọi bài toán [28]. Early Stopping là một ví dụ về việc áp dụng nguyên tắc này trong thực tế.

2.3.6 Tìm kiếm Optimal Thresholds

Trong phân loại đa nhãn với đầu ra là xác suất từ sigmoid function, ngưỡng mặc định (threshold) để chuyển đổi xác suất thành nhãn nhị phân thường là 0.5. Tuy nhiên, với dữ liệu mất cân bằng, ngưỡng 0.5 không phải lúc nào cũng tối ưu.

Đề tài áp dụng kỹ thuật tìm kiếm optimal threshold riêng biệt cho từng lớp:

$$\theta_c^* = \arg \max_{\theta \in [0.1, 0.9]} F1_c(\theta) \tag{2.48}$$

Với mỗi lớp c, duyệt qua các giá trị threshold từ 0.1 đến 0.9 với bước nhảy 0.01 và chọn threshold tối ưu dựa trên F1-score cao nhất. Phương pháp này cho phép mỗi lớp có một ngưỡng riêng phù hợp với phân phối và đặc điểm của lớp đó.

Kết quả cho thấy các optimal thresholds thường cao hơn 0.5 đáng kể (từ 0.62 đến 0.89), cho thấy mô hình có xu hướng đưa ra xác suất khá cao cho cả các mẫu không độc hại. Việc tăng threshold giúp lọc bớt các false positives và cải thiện precision.

Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn threshold phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến ROC curve và các độ đo hiệu suất khác nhau [15].

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Tập dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là Jigsaw Toxic Comment Classification Challenge từ Kaggle [10], một trong những cuộc thi phổ biến và được nghiên cứu rộng rãi nhất trong lĩnh vực phân loại văn bản độc hại. Cuộc thi này được tổ chức bởi Jigsaw (trước đây là Google Jigsaw) với mục tiêu phát triển các mô hình có khả năng nhận diện các loại bình luận độc hại khác nhau trên Wikipedia. Bộ dữ liệu này bao gồm các bình luận từ Wikipedia đã được gán nhãn thủ công bởi các chuyên gia kiểm duyệt theo 6 loại độc hại khác nhau. Tổng cộng có 159,571 bình luận trong tập dữ liệu, được chia theo tỷ lệ 80% cho tập huấn luyện (train) và 20% cho tập kiểm định (validation), sử dụng phương pháp stratified split để đảm bảo phân phối các lớp được giữ nguyên trong cả hai tập.

Phân tích chi tiết phân bố nhãn cho thấy mức độ mất cân bằng nghiêm trọng của dữ liệu. Lớp toxic có khoảng 15,000 mẫu dương tính (chiếm khoảng 9.4% tổng số mẫu), lớp obscene có khoảng 8,000 mẫu (5.0%), và lớp insult có khoảng 7,600 mẫu (4.8%). Các lớp thiểu số đặc biệt nghiêm trọng: severe_toxic chỉ có khoảng 1,600 mẫu (1.0%), identity_hate có khoảng 1,400 mẫu (0.9%), và đặc biệt threat chỉ có khoảng 320 mẫu (0.2%) - một con số rất nhỏ so với tổng thể 159,571 bình luận. Sự mất cân bằng này đặt ra thách thức lớn cho việc huấn luyện mô hình, buộc phải áp dụng các kỹ thuật xử lý mất cân bằng như Class-Weighted Loss để đảm bảo mô hình không bỏ qua hoàn toàn các lớp thiểu số. Hiện tượng này cũng đã được nghiên cứu trong nhiều bài toán phân loại văn bản khác, như đã được báo cáo bởi các nghiên cứu trước đây về hate speech detection [1].

Tập validation chứa 31,915 bình luận, được sử dụng để đánh giá hiệu suất của mô hình và thực hiện các thử nghiệm như tìm kiếm optimal thresholds. Tập test chứa khoảng 153,000 bình luận không có nhãn, được sử dụng để tạo file submission cho cuộc thi Kaggle. Việc đánh giá trên tập validation thay vì tập test (không có ground truth) là cách tiếp cận tiêu chuẩn trong nghiên cứu, cho phép so sánh và phân tích chi tiết hiệu suất của từng mô hình. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng validation set để đánh giá và lựa chọn mô hình, tránh để test set ảnh hưởng đến quá trình phát triển mô hình [27].

3.2 Cấu hình thử nghiệm

Để đảm bảo tính công bằng trong việc so sánh giữa các mô hình, tất cả các mô hình đều được huấn luyện với cùng bộ siêu tham số (hyperparameters) và chỉ khác nhau về kiến trúc mô hình. Các siêu tham số chính bao gồm: kích thước batch

(batch size) = 64, tốc độ học (learning rate) = 2×10^{-3}, số epochs tối đa = 10 với Early Stopping (patience = 2), độ dài tối đa chuỗi (max length) = 128 tokens, chiều embedding (embedding dimension) = 128, chiều hidden state (hidden size) = 256, số lớp RNN (num layers) = 2, và tỷ lệ dropout = 0.3. Ngoài ra, các tham số tối ưu hóa bao gồm weight decay = 10^{-4}, warmup ratio = 0.1, và gradient clip max norm = 1.0. Việc sử dụng chung hyperparameters cho tất cả các mô hình là một phương pháp phổ biến trong nghiên cứu so sánh các kiến trúc mô hình, như được áp dụng trong nhiều nghiên cứu trước đây [5], [6].

Toàn bộ quá trình huấn luyện được thực hiện trên GPU NVIDIA H100 với sự hỗ trợ của PyTorch [14]. Việc sử dụng GPU cho phép huấn luyện nhanh hơn đáng kể so với CPU, đặc biệt quan trọng khi huấn luyện với full dataset gồm 159,571 mẫu và 10 epochs. Thời gian huấn luyện cho mỗi epoch dao động từ khoảng 2 phút (Transformer, do xử lý song song) đến khoảng 9-10 phút (BiLSTM, Attention BiLSTM, RCNN, do xử lý tuần tự hai hướng). Tổng thời gian huấn luyện cho tất cả 7 mô hình (mỗi mô hình 10 epochs) vào khoảng 8-10 giờ trên GPU NVIDIA H100.

Tokenizer được sử dụng là AutoTokenizer từ thư viện Hugging Face Transformers [11] với pre-trained model "distilbert-base-uncased". DistilBERT là phiên bản nhỏ gọn hơn và nhanh hơn BERT, được huấn luyện trên 6GB dữ liệu văn bản tiếng Anh, do đó tokenizer của nó có khả năng tách từ thông minh, hiểu được subword tokenization (tách từ thành các tiền tố và hậu tố) giúp xử lý tốt các từ hiếm gặp và từ không có trong từ điển. Sanh và cộng sự (2019) đã chứng minh rằng DistilBERT đạt 97% hiệu suất của BERT trên benchmark GLUE trong khi chỉ sử dụng 60% tham số và thời gian inference nhanh hơn 60% [12].

Cấu hình chi tiết cho từng mô hình như sau: LSTM và Attention LSTM sử dụng 2 lớp LSTM với hidden size 256, embedding dimension 128; BiLSTM và Attention BiLSTM sử dụng 2 lớp forward LSTM và 2 lớp backward LSTM với hidden size 256 mỗi hướng; GRU sử dụng 2 lớp GRU với hidden size 256; RCNN sử dụng 2 lớp BiLSTM với hidden size 256 mỗi hướng, fusion layer với tanh activation; Transformer sử dụng 2 encoder layers, 8 attention heads, feed-forward dimension 512. Tất cả các mô hình đều sử dụng dropout 0.3 trước lớp fully connected cuối cùng.

3.3 So sánh tổng quan các mô hình

Kết quả huấn luyện và đánh giá trên tập validation cho thấy sự khác biệt đáng kể giữa các mô hình. Bảng tổng hợp dưới đây trình bày các chỉ số hiệu suất chính của tất cả 7 mô hình đã được thử nghiệm. Các độ đo được sử dụng bao gồm ROC-AUC

(diện tích dưới đường cong ROC), F1 Macro (trung bình F1 của các lớp), và F1 Micro (F1 tính tổng hợp trên tất cả các mẫu và nhãn), được tính toán dựa trên các công thức đã được phân tích chi tiết bởi Sokolova và Lapalme (2009) trong nghiên cứu về các độ đo cho bài toán phân loại [16].

Bảng 3.1: So sánh hiệu suất các mô hình trên tập validation (159,571 mẫu)

Mô hình	**Tham số (M)**	**Best Val AUC**	**Final AUC**	**F1 Macro**	**F1 Micro**
Attention BiLSTM	6.5	0.9834	0.9830	0.6311	0.7324
RCNN	6.6	**0.9839**	0.9831	0.5689	0.6655
Attention LSTM	4.9	0.9812	0.9791	0.5650	0.6820
BiLSTM	6.3	0.9800	0.9776	0.5520	0.6689
LSTM	4.8	0.9786	0.9761	0.5459	0.6658
GRU	4.6	0.9805	0.9793	0.5146	0.6198
Transformer	4.3	0.9789	0.9788	0.4625	0.5947

Phân tích kết quả cho thấy RCNN đạt ROC-AUC cao nhất (0.9839), cho thấy khả năng xếp hạng các mẫu dương tính cao hơn các mẫu âm tính của mô hình này là tốt nhất. Attention BiLSTM đứng thứ hai với AUC 0.9834 nhưng đạt F1 Macro cao nhất (0.6311), cho thấy sự cân bằng tốt nhất giữa precision và recall. Sự khác biệt về độ đo này là do AUC đo khả năng phân biệt ở mọi thresholds, trong khi F1 Macro tập trung vào hiệu suất tại các thresholds tối ưu cho từng lớp. Fawcett (2006) đã giải thích chi tiết sự khác biệt giữa AUC và các độ đo threshold-dependent như F1-score [15].

Các mô hình không có attention (BiLSTM, LSTM, GRU) và Transformer đều có F1 Macro thấp hơn đáng kể so với các mô hình có attention (Attention BiLSTM, Attention LSTM). Cụ thể, Attention BiLSTM (0.6311) cao hơn BiLSTM (0.5520) khoảng 0.08 điểm, và Attention LSTM (0.5650) cao hơn LSTM (0.5459) khoảng 0.02 điểm. Điều này chứng minh rằng cơ chế attention thực sự mang lại lợi ích đáng kể trong bài toán phân loại bình luận độc hại, cho phép mô hình tập trung vào các từ quan trọng và cải thiện khả năng phân biệt giữa các trường hợp thực sự thuộc nhãn và các trường hợp dương tính giả. Kết quả này nhất quán với các nghiên cứu trước đây về hiệu quả của attention trong các bài toán NLP [13], [22].

Đáng chú ý, GRU với ít tham số nhất (4.6 triệu) và huấn luyện nhanh hơn các mô hình BiLSTM đạt F1 Macro thấp nhất trong nhóm RNN (0.5146), cho thấy thiết kế đơn giản hóa của GRU có thể không đủ để nắm bắt các phụ thuộc phức tạp trong bài toán này. Transformer với kiến trúc hoàn toàn khác (không có recurrence) đạt F1 Macro thấp nhất (0.4625), có thể do cần nhiều dữ liệu và thời gian huấn luyện hơn để đạt hiệu suất tối ưu. Kết quả này cho thấy rằng kiến trúc phù hợp cho một

bài toán cụ thể cần được lựa chọn dựa trên đặc điểm của bài toán và dữ liệu, như đã được nhấn mạnh trong nghiên cứu về "No Free Lunch Theorems" của Wolpert (1997) [28].

3.4 Kết quả chi tiết từng mô hình

3.4.1 Kết quả của RCNN

RCNN đạt ROC-AUC cao nhất trong tất cả các mô hình (0.9839) và chi tiết hiệu suất trên từng lớp cho thấy mô hình này đặc biệt xuất sắc trong việc nhận diện obscene comments với F1-score 0.81 (precision 0.76, recall 0.87) và threat comments với F1-score 0.43 (precision 0.29, recall 0.79). Điểm mạnh của RCNN nằm ở kiến trúc kết hợp ngữ cảnh hai chiều từ BiLSTM với embedding gốc, tạo ra biểu diễn phong phú cho mỗi từ, sau đó sử dụng max pooling để trích xuất các đặc trưng nổi bật nhất. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN trong bài báo tại AAAI và chứng minh hiệu quả của nó trên nhiều bài toán phân loại văn bản [5]. Cách tiếp cận này đặc biệt hiệu quả khi các từ độc hại (như "fuck", "shit" cho obscene) xuất hiện rõ ràng và max pooling có thể dễ dàng chọn ra chúng.

Tuy nhiên, RCNN cũng có hạn chế rõ rệt trên các lớp thiểu số: severe_toxic chỉ đạt precision 0.25 với recall 0.96 (F1 = 0.40), và identity_hate chỉ đạt precision 0.20 với recall 0.85 (F1 = 0.32). Precision thấp trên các lớp này cho thấy mô hình có xu hướng dự đoán quá nhiều, tạo ra nhiều false positives. Nguyên nhân có thể là do max pooling chỉ chọn giá trị lớn nhất, có thể bỏ qua các tín hiệu quan trọng trong các trường hợp không có từ khóa rõ ràng. Khi áp dụng optimal thresholds, F1 Macro tăng lên 0.6328 và F1 Micro tăng lên 0.7428, cho thấy việc tinh chỉnh threshold vẫn mang lại cải thiện đáng kể.

Một điểm cần lưu ý là RCNN có số lượng tham số nhiều nhất (6.6 triệu) trong số tất cả các mô hình, đòi hỏi nhiều bộ nhớ GPU và thời gian huấn luyện lâu hơn (khoảng 9 phút mỗi epoch trên H100 GPU). Việc sử dụng BiLSTM 2 lớp để tạo ngữ cảnh hai chiều cộng với fusion layer tạo ra chi phí tính toán cao hơn so với các mô hình khác. Tuy nhiên, với ROC-AUC cao nhất, RCNN vẫn là một lựa chọn tốt cho các ứng dụng cần ưu tiên khả năng phân biệt tổng thể.

3.4.2 Kết quả của Attention BiLSTM

Attention BiLSTM đạt F1 Macro cao nhất (0.6311 với optimal thresholds = 0.6503) và ROC-AUC thứ hai (0.9834), cho thấy đây là mô hình có sự cân bằng tốt nhất giữa các độ đo hiệu suất. Kết quả chi tiết cho thấy mô hình đạt được precision cao hơn đáng kể so với RCNN ở hầu hết các lớp, đặc biệt là toxic (0.67 vs 0.62), severe_toxic (0.29 vs 0.25), và identity_hate (0.32 vs 0.20). Đồng thời, Attention BiLSTM vẫn duy trì recall ở mức tương đối cao cho hầu hết các lớp, với recall cao

nhất ở obscene (0.94) và insult (0.92). Sự kết hợp giữa BiLSTM và attention đã được nghiên cứu trước đó bởi Yang và cộng sự (2016) trong Hierarchical Attention Networks và đã chứng minh hiệu quả trên nhiều bài toán phân loại văn bản [22].

Điểm nổi bật của Attention BiLSTM là khả năng đạt F1 cao nhất trên lớp threat (0.48 với optimal thresholds) so với tất cả các mô hình khác. Điều này cho thấy cơ chế attention giúp mô hình học được cách nhận diện các mẫu đe dọa ngay cả khi chúng không chứa từ khóa rõ ràng, bằng cách tổng hợp thông tin từ nhiều phần của câu. Tuy nhiên, recall cho threat vẫn chỉ ở mức 0.68, thấp hơn nhiều so với các lớp khác, cho thấy vẫn còn khó khăn trong việc nhận diện tất cả các threat comments trong tập dữ liệu. Nguyên nhân có thể là do số lượng mẫu threat quá ít (chỉ 320 mẫu), khiến mô hình khó học đầy đủ các biểu diễn cần thiết.

Số lượng tham số của Attention BiLSTM là 6.5 triệu, gần tương đương với RCNN (6.6 triệu), và thời gian huấn luyện cũng tương tự (khoảng 9 phút mỗi epoch). Tuy nhiên, với F1 Macro cao hơn và precision tốt hơn trên các lớp thiểu số, Attention BiLSTM có thể được coi là lựa chọn tốt hơn cho các ứng dụng cần cân bằng precision và recall.

3.4.3 Kết quả của BiLSTM

BiLSTM không có attention đạt ROC-AUC 0.9800 và F1 Macro 0.5520 (với optimal thresholds = 0.5825), thấp hơn đáng kể so với Attention BiLSTM. Kết quả này xác nhận vai trò quan trọng của cơ chế attention trong việc cải thiện hiệu suất phân loại. BiLSTM sử dụng Global Average Pooling thay vì attention pooling, nghĩa là tất cả các time steps được tính bình quân với trọng số bằng nhau, khiến các từ ít quan trọng (như "the", "a", "is") cũng đóng góp ngang bằng với các từ quan trọng trong biểu diễn cuối cùng. Graves và cộng sự (2012) đã nghiên cứu về việc sử dụng bidirectional RNN trong nhận dạng giọng nói và nhận thấy rằng việc xử lý hai chiều mang lại cải thiện đáng kể so với unidirectional RNN [9].

Mặc dù recall trung bình của BiLSTM khá cao (0.88), precision lại rất thấp (0.36), dẫn đến F1 Macro thấp. Điều này cho thấy BiLSTM có xu hướng dự đoán quá nhiều các trường hợp là độc hại, tạo ra nhiều false positives. Cụ thể, trên lớp threat BiLSTM chỉ đạt precision 0.13, thấp hơn rất nhiều so với Attention BiLSTM (0.37), cho thấy việc thiếu attention mechanism khiến mô hình không thể phân biệt tốt giữa các bình luận thực sự là threat và các bình luận có từ ngữ tương tự nhưng không mang ý nghĩa đe dọa. BiLSTM có 6.3 triệu tham số và huấn luyện nhanh hơn một chút so với Attention BiLSTM (do không có attention layer), nhưng khoảng cách về hiệu suất là đáng kể.

3.4.4 Kết quả của Transformer

Transformer đạt ROC-AUC 0.9789 nhưng F1 Macro thấp nhất trong tất cả các mô hình (0.4625 với optimal thresholds = 0.5593). Đây là kết quả đáng chú ý vì Transformer là kiến trúc hiện đại và mạnh mẽ nhất trong danh sách, được sử dụng rộng rãi trong các mô hình ngôn ngữ lớn. Vaswani và cộng sự (2017) đã giới thiệu Transformer trong bài báo "Attention Is All You Need" và chứng minh hiệu quả vượt trội trên bài toán dịch máy [6]. Tuy nhiên, kết quả trong đề tài này cho thấy Transformer cần được điều chỉnh cấu hình phù hợp để đạt hiệu suất tối ưu trên bài toán phân loại bình luận độc hại.

Nguyên nhân chính của hiệu suất thấp có thể bao gồm: thứ nhất, việc sử dụng embedding ngẫu nhiên (random initialization) thay vì pretrained embeddings như GloVe [17] hoặc FastText [18] khiến mô hình phải học tất cả các biểu diễn từ đầu, đòi hỏi nhiều dữ liệu và thời gian hơn; thứ hai, Transformer thường cần huấn luyện nhiều epochs hơn (20-50 epochs) để đạt hiệu suất tối ưu, trong khi thử nghiệm chỉ chạy 10 epochs; thứ ba, cấu hình 2 layers và 8 heads có thể chưa đủ mạnh để nắm bắt các phụ thuộc phức tạp trong bài toán này. Devlin và cộng sự (2019) đã chứng minh rằng việc sử dụng pre-trained BERT với fine-tuning mang lại cải thiện vượt bậc trên nhiều bài toán NLP [19].

Transformer có ưu điểm về tốc độ huấn luyện: mỗi epoch chỉ mất khoảng 2 phút (nhanh hơn 4-5 lần so với BiLSTM) do xử lý song song toàn bộ chuỗi thay vì tuần tự. Số lượng tham số cũng ít nhất (4.3 triệu). Nếu được huấn luyện với pretrained embeddings và nhiều epochs hơn, Transformer có tiềm năng đạt hiệu suất cao hơn.

3.5 Phân tích ảnh hưởng của các kỹ thuật tối ưu hóa

Các kỹ thuật tối ưu hóa đã được áp dụng trong đề tài đã đóng góp đáng kể vào việc cải thiện hiệu suất của tất cả các mô hình. So sánh với kết quả từ các thử nghiệm trước đó trên tập 10,000 mẫu, việc huấn luyện trên full dataset (159,571 mẫu) mang lại cải thiện vượt trội: ROC-AUC tăng từ khoảng 0.95 lên 0.98 (tăng khoảng 0.03 điểm), và F1 Macro tăng từ khoảng 0.43 lên 0.63 (tăng khoảng 0.20 điểm). Điều này chứng minh tầm quan trọng của việc huấn luyện trên dữ liệu đầy đủ để mô hình học được các biểu diện tốt. Goodfellow và cộng sự (2016) đã thảo luận trong cuốn sách "Deep Learning" rằng mạng nơ-ron học sâu thường cần lượng dữ liệu lớn để đạt hiệu suất tối ưu [21].

Class-Weighted Loss là kỹ thuật có ảnh hưởng lớn nhất đến hiệu suất trên các lớp thiểu số. Trước khi áp dụng class weights, mô hình gần như bỏ qua hoàn toàn các lớp như threat và identity_hate do số lượng mẫu quá ít. Sau khi áp dụng class weights với clipping ở mức 50, mô hình bắt đầu nhận diện được các mẫu từ các lớp

này, với recall dao động từ 0.68 đến 0.96 tùy mô hình. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng trong object detection và nhận thấy rằng việc giảm trọng số của các mẫu dễ phân loại giúp mô hình tập trung vào các mẫu khó [7]. Tuy nhiên, precision vẫn thấp ở các lớp thiểu số (0.11-0.37), cho thấy vẫn còn nhiều false positives và cần các cải tiến thêm như data augmentation hoặc pretrained embeddings.

AdamW với weight decay giúp ngăn overfitting và cải thiện khả năng tổng quát hóa của mô hình. Loshchilov và Hutter (2017) đã chứng minh rằng AdamW cải thiện đáng kể so với Adam thông thường trong việc regularization và đạt kết quả tốt hơn trên các benchmark [8]. Gradient clipping đảm bảo quá trình huấn luyện ổn định, đặc biệt quan trọng đối với các mô hình RNN. Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài [9]. Early Stopping đã ngăn chặn overfitting hiệu quả, với hầu hết các mô hình dừng ở epoch 4-7 thay vì chạy đủ 10 epochs, cho thấy mô hình đã đạt đến điểm tối ưu trước khi bắt đầu overfit.

Warmup Cosine LR Scheduler giúp quá trình huấn luyện ổn định và hội tụ tốt hơn. Khi quan sát learning rate curves, ta thấy learning rate tăng tuyến tính trong giai đoạn warmup (khoảng 200 steps đầu), sau đó giảm dần theo hàm cosine trong phần còn lại. Vaswani và cộng sự (2017) đã sử dụng warmup schedule trong huấn luyện Transformer và nhận thấy rằng nó giúp ổn định quá trình huấn luyện trong giai đoạn đầu [6]. Điều này cho phép mô hình khám phá rộng rãi không gian tham số ở giai đoạn đầu và fine-tune ở giai đoạn cuối.

Kỹ thuật tìm kiếm optimal thresholds riêng biệt cho từng lớp cũng mang lại cải thiện đáng kể. Các optimal thresholds thường cao hơn 0.5 đáng kể (từ 0.62 đến 0.89), cho thấy mô hình có xu hướng đưa ra xác suất khá cao cho cả các mẫu không độc hại. Việc tăng threshold giúp lọc bớt các false positives và cải thiện precision. Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn threshold phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến ROC curve và các độ đo hiệu suất khác nhau [15].

3.6 Nhận xét chung

Qua quá trình thử nghiệm và đánh giá 7 mô hình Deep Learning khác nhau trên bài toán phân loại bình luận độc hại, một số nhận xét quan trọng có thể được rút ra. Thứ nhất, cơ chế Attention thực sự mang lại lợi ích đáng kể trong bài toán này. Attention BiLSTM vượt trội hơn BiLSTM ở cả ROC-AUC lẫn F1 Macro, và sự cải thiện này đến từ khả năng tập trung vào các từ quan trọng của attention thay vì tính

bình quân tất cả các time steps như trong mean pooling truyền thống. Kết quả này nhất quán với các nghiên cứu trước đây của Bahdanau và cộng sự (2014) về hiệu quả của attention trong các bài toán NLP [13].

Thứ hai, kiến trúc Bi-directional (xử lý hai chiều) vượt trội hơn kiến trúc uni-directional trong hầu hết các trường hợp. BiLSTM đạt kết quả tốt hơn LSTM, và Attention BiLSTM đạt kết quả tốt hơn Attention LSTM. Điều này cho thấy việc nắm bắt ngữ cảnh cả trước và sau một từ là quan trọng trong việc hiểu ý nghĩa của văn bản, đặc biệt khi có các từ phủ định hoặc các mẫu phức tạp. Graves và cộng sự (2012) đã chứng minh rằng việc xử lý hai chiều mang lại cải thiện đáng kể trong nhận dạng giọng nói [9].

Thứ ba, việc xử lý dữ liệu mất cân bằng là yếu tố then chốt quyết định thành công của bài toán. Không có Class-Weighted Loss, mô hình gần như không thể nhận diện được các lớp thiểu số như threat (chỉ 0.2% dữ liệu). Với Class-Weighted Loss và clipping, recall cho tất cả các lớp đều đạt mức cao (0.68-0.97), cho thấy mô hình đã học được cách nhận diện các mẫu từ các lớp thiểu số. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng và nhận thấy rằng nguyên tắc giảm trọng số của các mẫu dễ phân loại mang lại cải thiện đáng kể [7].

Thứ tư, Precision vẫn là điểm yếu chung của tất cả các mô hình, đặc biệt trên các lớp thiểu số. Precision thấp nhất là ở threat (0.09-0.37) và identity_hate (0.11-0.32), cho thấy mô hình vẫn tạo ra nhiều false positives trên các lớp này. Nguyên nhân có thể là do số lượng mẫu quá ít khiến mô hình không thể học được các biểu diễn đủ tốt để phân biệt chính xác giữa các trường hợp thực sự và giả của các lớp thiểu số. Sokolova và Lapalme (2009) đã phân tích chi tiết về các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score là một độ đo cân bằng, nhưng precision và recall có thể thay đổi đáng kể tùy thuộc vào ngưỡng được chọn [16].

Thứ năm, Transformer với cấu hình hiện tại chưa đạt hiệu suất tốt như các mô hình RNN trong thử nghiệm này. Tuy nhiên, điều này không có nghĩa là Transformer không phù hợp cho bài toán này, mà có thể cần cấu hình khác (nhiều layers hơn, pretrained embeddings, nhiều epochs huấn luyện) để phát huy hết tiềm năng của kiến trúc. Vaswani và cộng sự (2017) đã huấn luyện Transformer với nhiều layers hơn và đạt hiệu suất vượt trội trên bài toán dịch máy [6].

Cuối cùng, sự kết hợp giữa RCNN (Bi-directional + embedding fusion + max pooling) và Attention BiLSTM (Bi-directional + attention) cho thấy hai hướng tiếp cận khác nhau để cải thiện biểu diễn: RCNN tập trung vào việc trích xuất các đặc trưng mạnh nhất (max pooling), trong khi Attention BiLSTM tập trung vào việc

tổng hợp có trọng số các đặc trưng. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN và nhận thấy rằng việc kết hợp RNN và CNN mang lại hiệu quả tốt trên bài toán phân loại văn bản [5]. Cả hai đều vượt trội hơn các mô hình cơ bản không có các cải tiến này.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Tổng kết

Đề tài đã hoàn thành việc nghiên cứu và cài đặt từ gốc bảy mô hình Deep Learning khác nhau để giải quyết bài toán phân loại đa nhãn bình luận độc hại trên bộ dữ liệu Jigsaw Toxic Comment Classification Challenge [10]. Tất cả các mô hình đều được cài đặt chi tiết từ các thành phần cơ bản nhất như LSTM Cell, GRU Cell, Self-Attention, Multi-Head Attention, Positional Encoding, và các pooling strategies, thể hiện sự hiểu biết sâu về kiến trúc bên trong của các mô hình học sâu hiện đại. Việc cài đặt từ gốc không chỉ giúp hiểu rõ cách các mô hình hoạt động mà còn cho phép tùy chỉnh và tối ưu hóa theo nhu cầu cụ thể của bài toán.

Kết quả nghiên cứu cho thấy mô hình RCNN đạt ROC-AUC cao nhất (0.9839), trong khi mô hình Attention BiLSTM đạt F1 Macro cao nhất (0.6503 với optimal thresholds), cho thấy hai mô hình này là những lựa chọn tốt nhất cho bài toán phân loại bình luận độc hại. RCNN (Recurrent Convolutional Neural Network) được đề xuất bởi Lai và cộng sự năm 2015 tại AAAI [5], kết hợp khả năng nắm bắt ngữ cảnh của BiLSTM với việc trích xuất đặc trưng bằng max pooling để đạt hiệu suất xếp hạng tốt nhất. Attention BiLSTM kết hợp xử lý hai chiều với cơ chế attention được nghiên cứu bởi Bahdanau và cộng sự (2014) [13] và Yang và cộng sự (2016) [22], đạt sự cân bằng tốt nhất giữa precision và recall.

Kiến trúc kết hợp giữa BiLSTM (cho ngữ cảnh hai chiều) với Attention (cho phép tập trung vào các từ quan trọng) đã chứng minh được hiệu quả vượt trội so với các mô hình cơ bản không có các cải tiến này. Cụ thể, Attention BiLSTM cải thiện F1 Macro lên khoảng 0.08 điểm so với BiLSTM (0.6311 vs 0.5520), một sự cải thiện đáng kể trong bài toán có dữ liệu mất cân bằng nghiêm trọng. Tương tự, Attention LSTM (0.5650) vượt trội hơn LSTM (0.5459) khoảng 0.02 điểm. Điều này chứng minh rằng cơ chế attention không chỉ cải thiện khả năng biểu diễn của mô hình mà còn giúp phân biệt tốt hơn giữa các trường hợp thực sự thuộc nhãn độc hại và các trường hợp dương tính giả.

Việc huấn luyện trên full dataset gồm 159,571 bình luận đã mang lại cải thiện vượt trội so với việc huấn luyện trên tập con 10,000 mẫu: ROC-AUC tăng khoảng 0.03 điểm (từ 0.95 lên 0.98) và F1 Macro tăng khoảng 0.20 điểm (từ 0.43 lên 0.63). Điều này khẳng định tầm quan trọng của việc sử dụng dữ liệu đầy đủ trong các bài toán Deep Learning để mô hình có thể học được các biểu diễn tốt và tổng quát hóa hiệu quả. Goodfellow và cộng sự (2016) đã thảo luận chi tiết trong cuốn sách "Deep Learning" rằng mạng nơ-ron học sâu thường cần lượng dữ liệu lớn để đạt

hiệu suất tối ưu, và việc thiếu dữ liệu có thể dẫn đến underfitting hoặc overfitting nghiêm trọng [21].

Các kỹ thuật tối ưu hóa huấn luyện đã đóng vai trò quan trọng trong việc đạt được kết quả tốt. Class-Weighted Loss với clipping ở mức 50 đã giúp mô hình nhận diện được các lớp thiểu số (threat, severe_toxic, identity_hate) mà trước đó gần như bị bỏ qua hoàn toàn. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng, và nguyên tắc tương tự đã được áp dụng thành công trong đề tài này với việc sử dụng pos_weight trong BCEWithLogitsLoss [7]. AdamW với weight decay giúp ngăn overfitting, được nghiên cứu và chứng minh hiệu quả bởi Loshchilov và Hutter (2017) [8]. Gradient clipping đảm bảo quá trình huấn luyện ổn định cho các mô hình RNN [9], warmup cosine LR scheduler giúp hội tụ tốt hơn [6], và early stopping ngăn chặn overfitting hiệu quả. Kỹ thuật tìm kiếm optimal thresholds riêng biệt cho từng lớp cũng mang lại cải thiện đáng kể, giúp cải thiện precision mà vẫn duy trì recall ở mức cao.

Thách thức chính trong bài toán này vẫn là sự mất cân bằng nghiêm trọng giữa các lớp. Mặc dù đã đạt recall cao (0.68-0.97) cho tất cả các lớp nhờ Class-Weighted Loss, precision vẫn thấp ở các lớp thiểu số (0.09-0.37), dẫn đến nhiều false positives. Hiện tượng này là phổ biến trong các bài toán với dữ liệu mất cân bằng nghiêm trọng, như đã được báo cáo trong nhiều nghiên cứu trước đây [16]. Điều này cho thấy cần có các phương pháp bổ sung như data augmentation, pretrained embeddings, hoặc các kỹ thuật xử lý mất cân bằng nâng cao hơn để cải thiện thêm khả năng phân biệt của mô hình.

Nhìn tổng thể, đề tài đã hoàn thành mục tiêu đề ra là cài đặt và so sánh các mô hình Deep Learning từ gốc cho bài toán phân loại bình luận độc hại, đồng thời áp dụng các kỹ thuật tối ưu hóa tiên tiến để đạt hiệu suất cao. Kết quả nghiên cứu cung cấp một cái nhìn toàn diện về ưu nhược điểm của từng kiến trúc mô hình, đồng thời đề xuất các hướng cải tiến cho nghiên cứu trong tương lai.

4.2 Hướng phát triển

Dựa trên kết quả nghiên cứu và các hạn chế còn tồn tại, một số hướng phát triển được đề xuất cho các nghiên cứu tiếp theo.

Hướng phát triển đầu tiên và quan trọng nhất là sử dụng Pretrained Embeddings thay vì embedding ngẫu nhiên. Các pretrained embeddings như GloVe (Global Vectors for Word Representation) được phát triển bởi Pennington và cộng sự năm 2014 [17], FastText được phát triển bởi Mikolov và cộng sự năm 2013 [18], hoặc ELMo (Embeddings from Language Models) đã được huấn luyện trên các corpus văn bản lớn, cung cấp các biểu diễn từ có chất lượng cao và có thể nắm bắt được

các quan hệ ngữ nghĩa phong phú. Việc khởi tạo embedding layer bằng pretrained embeddings thay vì random initialization có thể giúp mô hình hội tụ nhanh hơn và đạt hiệu suất cao hơn, đặc biệt trên các lớp thiểu số. Kim (2014) đã nghiên cứu về việc sử dụng pretrained word vectors trong CNN cho phân loại câu và nhận thấy rằng pretrained embeddings cải thiện đáng kể hiệu suất [29].

Hướng phát triển thứ hai là Fine-tuning Class Weights. Thay vì sử dụng tỷ lệ đơn thuần (neg_count / pos_count) với clipping cố định, có thể thử nghiệm với các phương pháp tìm kiếm trọng số tối ưu như grid search hoặc Bayesian optimization trên validation set. Ngoài ra, Focal Loss được thiết kế đặc biệt cho dữ liệu mất cân bằng bởi Lin và cộng sự năm 2017 [7], giảm trọng số của các mẫu dễ phân loại và tập trung vào các mẫu khó, có thể là một lựa chọn thay thế worth exploring. Focal Loss đã được chứng minh hiệu quả trong nhiều bài toán object detection và có tiềm năng cải thiện hiệu suất trên các lớp thiểu số trong bài toán phân loại bình luận độc hại.

Hướng thứ ba là xây dựng Ensemble Model kết hợp predictions từ nhiều mô hình khác nhau. Dựa trên kết quả nghiên cứu, sự kết hợp giữa Attention BiLSTM (F1 Macro cao nhất) và RCNN (AUC cao nhất) có thể mang lại hiệu suất tốt hơn so với bất kỳ mô hình đơn lẻ nào. Ensemble có thể được thực hiện bằng nhiều cách như voting, averaging probabilities, hoặc stacking với meta-learner. Nghiên cứu về ensemble methods trong học máy đã chứng minh rằng việc kết hợp nhiều mô hình thường mang lại kết quả tốt hơn so với bất kỳ mô hình đơn lẻ nào, như đã được thảo luận trong cuốn sách "Deep Learning" của Goodfellow và cộng sự [21].

Hướng thứ tư là Data Augmentation cho các lớp thiểu số. Với threat chỉ có khoảng 320 mẫu, việc tăng cường dữ liệu cho lớp này có thể giúp mô hình học tốt hơn. Các kỹ thuật augmentation phù hợp cho văn bản bao gồm: back-translation (dịch sang ngôn ngữ khác rồi dịch lại), synonym replacement (thay thế từ bằng từ đồng nghĩa), random insertion/deletion/swap, và paraphrasing sử dụng các mô hình language model. Kovács và Sarlós (2019) đã nghiên cứu về việc tạo synthetic data cho các mô hình sequence-to-sequence và nhận thấy rằng data augmentation có thể cải thiện đáng kể hiệu suất của mô hình trên các lớp thiểu số [30].

Hướng thứ năm là Hyperparameter Tuning chi tiết hơn cho từng mô hình. Trong đề tài này, tất cả các mô hình sử dụng chung bộ hyperparameters, nhưng mỗi kiến trúc có đặc điểm riêng và có thể cần các thiết lập khác nhau. Cụ thể, Transformer có thể cần nhiều layers hơn (4-6 thay vì 2), learning rate nhỏ hơn (1e-4 thay vì 2e-3), và nhiều epochs hơn (20-50 thay vì 10) để đạt hiệu suất tối ưu. Vaswani và cộng sự (2017) đã huấn luyện Transformer với 6 encoder layers và đạt hiệu suất

vượt trội trên bài toán dịch máy [6]. Grid search hoặc Bayesian optimization có thể được sử dụng để tìm bộ hyperparameters tốt nhất cho từng mô hình.

Hướng thứ sáu là sử dụng Transformer-based pretrained models như BERT, RoBERTa, hoặc DistilBERT cho feature extraction hoặc fine-tuning. Mặc dù trong đề tài chỉ sử dụng tokenizer của DistilBERT, việc fine-tune toàn bộ pre-trained model có thể mang lại cải thiện vượt bậc về hiệu suất, vì các mô hình này đã được huấn luyện trên hàng triệu văn bản và nắm bắt được các biểu diễn ngôn ngữ rất tinh tế. Devlin và cộng sự (2019) đã giới thiệu BERT và chứng minh rằng việc pre-training trên large corpus followed by fine-tuning mang lại kết quả ấn tượng trên nhiều bài toán NLP [19]. Tuy nhiên, điều này đi ngược lại mục tiêu của đề tài là cài đặt từ gốc các mô hình RNN, nên đây có thể là hướng phát triển cho các nghiên cứu khác tập trung vào hiệu suất tối đa.

Cuối cùng, việc triển khai ứng dụng thực tế với mô hình đã chọn (Attention BiLSTM hoặc RCNN) để kiểm duyệt bình luận độc hại trên các nền tảng mạng xã hội là một hướng phát triển có ý nghĩa thực tiễn cao. Ứng dụng có thể bao gồm giao diện người dùng để nhập bình luận và nhận kết quả phân loại theo thời gian thực, tích hợp API để các nền tảng khác có thể sử dụng, và dashboard để quản lý và giám sát các bình luận đã được phân loại. Việc interpretability của mô hình cũng có thể được cải thiện bằng cách visualize attention weights, giúp người dùng hiểu tại sao một bình luận được phân loại là độc hại. Ribeiro và cộng sự (2016) đã nghiên cứu về việc giải thích dự đoán của các bộ phân loại và nhấn mạnh tầm quan trọng của interpretability trong các ứng dụng thực tế [31].

TÀI LIỆU THAM KHẢO

[1] N. Djuric, J. Zhou, R. Morris, M. Grbavic, V. Duk, and Bosch, "Hate speech detection with comment embeddings," pp. 1–2, 2015. DOI: 10.1145/2740908.2742760 [Online]. Available: https://doi.org/10.1145/2740908.2742760

[2] E. S. Chung, J. K. Ousterhout, and A. W. V. Sastry, "Automatic detection of harassment on Twitter," 2012.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735 [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[4] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," pp. 1724–1734, 2014. DOI: 10.3115/v1/D14-1179 [Online]. Available: https://doi.org/10.3115/v1/D14-1179

[5] S. Lai, L. Xu, K. Liu, and J. Zhao, "Recurrent convolutional neural networks for text classification," pp. 2267–2273, 2015. [Online]. Available: https://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9745

[6] A. Vaswani et al., "Attention is all you need," vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," pp. 2980–2988, 2017. DOI: 10.1109/ICCV.2017.324 [Online]. Available: https://doi.org/10.1109/ICCV.2017.324

[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, arXiv:1711.05101. [Online]. Available: https://arxiv.org/abs/1711.05101

[9] A. Graves, A. rahman Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," pp. 6645–6649, 2013. DOI: 10.1109/ICASSP.2013.6638947 [Online]. Available: https://doi.org/10.1109/ICASSP.2013.6638947

[10] Jigsaw and Google, "Jigsaw toxic comment classification challenge," 2018. [Online]. Available: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

[11] T. Wolf et al., "Transformers: State-of-the-art natural language processing," pp. 38–45, 2020. DOI: 10.18653/v1/2020.emnlp-demos.6 [Online]. Available: https://doi.org/10.18653/v1/2020.emnlp-demos.6

[12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, arXiv:1910.01108. [Online]. Available: https://arxiv.org/abs/1910.01108

[13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *3rd International Conference on Learning Representations (ICLR)*, 2014, arXiv:1409.0473. [Online]. Available: https://arxiv.org/abs/1409.0473

[14] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," vol. 32, pp. 8026–8037, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/b872173a90933b87a9888a3a0602f84c-Abstract.html

[15] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010 [Online]. Available: https://doi.org/10.1016/j.patrec.2005.10.010

[16] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. DOI: 10.1016/j.ipm.2009.03.002 [Online]. Available: https://doi.org/10.1016/j.ipm.2009.03.002

[17] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," pp. 1532–1543, 2014. DOI: 10.3115/v1/D14-1162 [Online]. Available: https://doi.org/10.3115/v1/D14-1162

[18] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, arXiv:1301.3781. [Online]. Available: https://arxiv.org/abs/1301.3781

[19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," vol. 1, pp. 4171–

4186, 2019. DOI: 10.18653/v1/N19-1423 [Online]. Available: https://doi.org/10.18653/v1/N19-1423

[20] R. Baeza-Yates and B. Ribeiro-Neto, "Modern information retrieval," 1999.

[21] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning," 2016. [Online]. Available: https://www.deeplearningbook.org/

[22] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," pp. 1480–1489, 2016. DOI: 10.18653/v1/N16-1174 [Online]. Available: https://doi.org/10.18653/v1/N16-1174

[23] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980. [Online]. Available: https://arxiv.org/abs/1412.6980

[25] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html

[26] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient BackPropagation," pp. 9–50, 1998. DOI: 10.1007/3-540-49430-8_2

[27] S. Raschka, "Model evaluation, model selection, and algorithm selection in machine learning," *arXiv preprint arXiv:1811.12808*, 2018. [Online]. Available: https://arxiv.org/abs/1811.12808

[28] D. H. Wolpert, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, 1997. DOI: 10.1109/4235.585893

[29] b. P. p. y. a. D. p. A. d. v. u. h. Yoon Kim title = Convolutional Neural Networks for Sentence Classification,

[30] G. Kovács and P. Sarlós, "Man is like a drug: Creating synthetic data for sequence-to-sequence models," pp. 56–65, 2018. DOI: 10.18653/v1/K18-1006 [Online]. Available: https://doi.org/10.18653/v1/K18-1006

[31] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should i trust you?": Explaining the predictions of any classifier," pp. 1135–1144, 2016. DOI: 10.

1145/2939672.2939778 [Online]. Available: https://doi.org/10.1145/2939672.2939778